

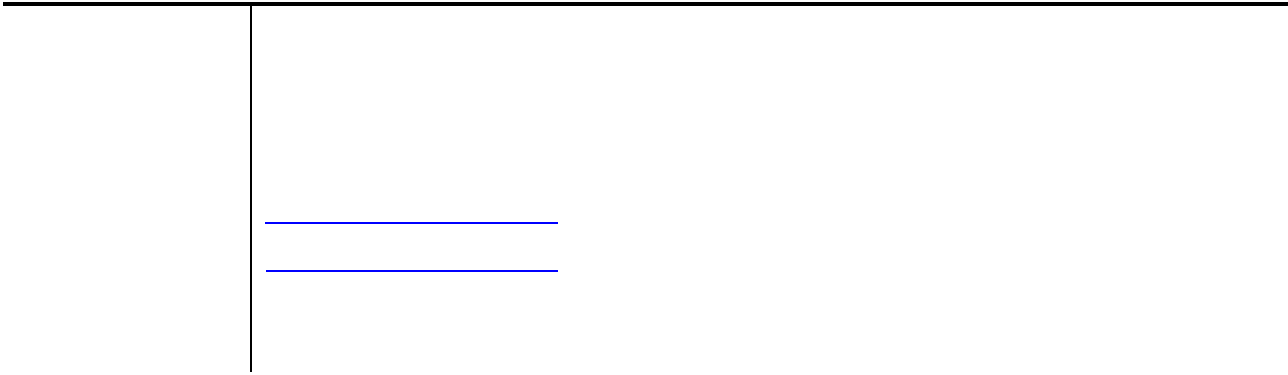
Autodesk® **Scaleform®**

Copyright Notice

Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved.

Disclaimer



1. はじめに

2. DrawText サンプル コードの概要

1. はじめに

- *Gfx::DrawTextManager*

- *Gfx::DrawText*

1.1. Gfx::DrawTextManager

1. DrawTextManager();

```
Ptr<DrawTextManager> pdm = *new DrawTextManager();
```

2. DrawTextManager(MovieDef* pmovieDef);

```
Ptr<Gfx::MovieDef> pmovieDef = *mLoader.CreateMovie("drawtext_fonts.swf",  
Loader::LoadAll);  
Ptr<Gfx::DrawTextManager> pDm2 = *new DrawTextManager(pmovieDef);
```

3. DrawTextManager(Gfx::Loader* ploader);

```

Gfx::Loader mLoader;
..
Ptr<Gfx::DrawTextManager> pDm1 = *new DrawTextManager(&mLoader);

```

1.1.1. Gfx::DrawText の作成

- DrawText* CreateText(const char* putf8Str, const RectF& viewRect, const TextParams* ptxtParams = NULL, unsigned depth = ~0u);
- DrawText* CreateText(const wchar_t* pwstr, const RectF& viewRect, const TextParams* ptxtParams = NULL, unsigned depth = ~0u);
- DrawText* CreateText(const String& str, const RectF& viewRect, const TextParams* ptxtParams = NULL, unsigned depth = ~0u);

```

putf8str
pwstr
str

```

```

viewRect

```

```

ptxtParams

```

```

struct TextParams
{
    Color                TextColor;
    DrawText::Alignment  HAlignment;
    DrawText::VAlignment VAlignment;
    DrawText::FontStyle  FontStyle;
    float                FontSize;
    String               FontName;
    bool                 Underline;
}

```

```

        bool                Multiline;
        bool                WordWrap;
};

```

```

    TextColor :

```

```

    HAlignment :                                DrawText::Align_Left
        DrawText::Align_Right、DrawText::Align_Center、DrawText::Align_Justify

```

```

    VAlignment                                DrawText::VAlign_Top
        DrawText::VAlign_Bottom DrawText::VAlign_Center

```

```

    FontStyle :
        DrawText::Normal DrawText::Bold DrawText::Italic
    DrawText::BoldItalic DrawText::ItalicBold

```

```

    FontSize :

```

```

    FontName

```

```

    Underline

```

```

    Multiline

```

```

    WordWrap

```

```

ptxtParams

```

```

void SetDefaultTextParams(const TextParams& params);
const TextParams& GetDefaultTextParams() const;

```

```

// 指定されたHTMLを使ってGfx::DrawTextオブジェクトを作成し初期化する。

```

```

DrawText* CreateHtmlText(const char* putf8Str, const RectF& viewRect,
                        const TextParams* ptxtParams = NULL, unsigned depth = ~0u);
DrawText* CreateHtmlText(const wchar_t* pwstr, const RectF& viewRect,
                        const TextParams* ptxtParams = NULL, unsigned depth = ~0u);
DrawText* CreateHtmlText(const String& str, const RectF& viewRect,

```



```
const TextParams* ptxtParams = NULL, unsigned depth =  
~0u);
```

```
//  
String str("String No 2");  
DrawTextManager::TextParams params;  
params.FontName    = "Symbol";  
params.FontSize    = 30;  
params.FontStyle   = DrawText::Italic;  
params.Multiline   = false;  
params.HAlignment  = DrawText::Align_Right;  
params.VAlignment  = DrawText::VAlign_Bottom;  
  
Ptr<DrawText> ptxt;  
ptxt = *pdm->CreateText(str, RectF(20, 300, 400, 700), &params);  
  
//  
Ptr<DrawText> ptxt;  
ptxt = *pdm->CreateHtmlText("<p><FONT size='20'>AB  
    <b>singleline</b><i> CD</i></p>",  
    RectF(20, 300, 400, 700));
```

1.1.2. テキストのレンダリング

1.1.3. テキスト範囲の計測

```
SizeF GetTextExtent(const char* putf8Str, float width = 0,  
    const TextParams* ptxtParams = 0);  
SizeF GetTextExtent(const wchar_t* pwstr, float width = 0,  
    const TextParams* ptxtParams = 0);  
SizeF GetTextExtent(const String& str, float width = 0,  
    const TextParams* ptxtParams = 0);  
  
SizeF GetHtmlTextExtent(const char* putf8Str, float width = 0,
```

```
        const TextParams* ptxtParams = 0);
SizeF GetHtmlTextExtent(const wchar_t* pwstr, float width = 0,
        const TextParams* ptxtParams = 0);
SizeF GetHtmlTextExtent(const String& str, float width = 0,
        const TextParams* ptxtParams = 0);
```

'width'

ptxtParams

'ptxtParams'

```
//
String str("String No 2");
```

1.2. Gfx::DrawText

1.2.1. プレーン テキスト メソッドの設定と取得

‘lengthInBytes’
‘lengthInChars’

```
void SetText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));  
void SetText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));  
void SetText(const String& str);
```

```
String GetText() const;
```

1.2.2. HTML テキスト メソッドの設定と取得

‘lengthInBytes’ ‘lengthInChars’

```
void SetHtmlText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));  
void SetHtmlText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));  
void SetHtmlText(const String& str);
```

```
String GetHtmlText() const;
```

1.2.3. テキスト フォーマットを設定するメソッド

```
void SetColor(Color c, UInt startPos = 0, UInt endPos = UInt(-1));
```

```
    'startPos' 'endPos'
```

```
void SetFont (const char* pfontName, UInt startPos = 0, UInt endPos = UInt(-1))
```

```
    'startPos' 'endPos'
```

```
void SetFontSize(float fontSize, UInt startPos = 0, UInt endPos = UInt(-1))
```

```
    'startPos' 'endPos'
```

```
void SetFontStyle(FontStyle, UInt startPos = 0, UInt endPos = UInt(-1));
```

```
enum FontStyle
```

```
{
```

```
    Normal,
```

```
    Bold,
```

```
    Italic,
```

```
    BoldItalic,
```

```
    ItalicBold = BoldItalic
```

```
};
```

```
    'startPos' 'endPos'
```

```
void SetUnderline(bool underline, UInt startPos = 0, UInt endPos = UInt(-1))
```

```
    'startPos' 'endPos'
```

```
void SetMultiline(bool multiline);
```

```
    'multiline'
```

```
bool IsMultiline() const;
```

```
    'false'
```

```
void SetWordWrap(bool wordWrap);
```

```
bool Wrap() const;
```

1.2.4. テキストの配置メソッド

```
enum Alignment
{
    Align_Left,
    Align_Default = Align_Left,
    Align_Right,
    Align_Center,
    Align_Justify
};
enum VAlignment
{
    VAlign_Top,
    VAlign_Default = VAlign_Top,
    VAlign_Center,
    VAlign_Bottom
};
```

```
void SetAlignment(Alignment);
```

```
Alignment    GetAlignment() const;
```

```
void SetVAlignment(VAlignment);
```

```
VAlignment    GetVAlignment() const;
```

1.2.5. テキストの位置と向きの変更

```
void SetRect(const RectF& viewRect);
```

```
RectF GetRect() const;
```

```
void SetMatrix(const Matrix& matrix);
```

```
const Matrix GetMatrix() const;
```

```
void SetCxform(const Cxform& cx);
```

```
const Cxform& GetCxform() const;
```

2. DrawText サンプル コードの概要

```
class FxPlayerApp : public FxPlayerAppBase
{
    public:
        FxPlayerApp();

        virtual bool          OnInit(Platform::ViewConfig& config);
        virtual void          OnUpdateFrame(bool needRepaint);

        Ptr<GFx::DrawTextManager> pDm1, pDm2;
        Ptr<GFx::DrawText>        ptxt11, ptxt12, ptxt21, ptxt22, ptxtImg,
                                pblurTxt, pglowTxt, pdropShTxt, pblurglowTxt;

        float   Angle;
        UInt32  Color;
};
```

方法 1

```
pDm1 = *new DrawTextManager(&mLoader);
```

```
pDm1->SetFontProvider(mLoader.GetFontProvider());
```

```
DrawTextManager::TextParams defParams =  
    pDm1->GetDefaultTextParams();  
defParams.TextColor = Color(0xF0, 0, 0, 0xFF); // red, alpha = 255  
defParams.FontName = "Arial";  
defParams.FontSize = 16;  
pDm1->SetDefaultTextParams(defParams);
```

```
ptxt11 = *pDm1->CreateText ("Plain text, red, Arial,  
                             16pts", RectF(20, 20, 500, 400));
```

```
// StringとTextParams  
String str(  
    "Scaleform is a light-weight high-performance rich media vector  
    graphics and user interface (UI) engine.");  
Gfx::DrawTextManager::TextParams params;  
params.FontName = "Arial";  
params.FontSize = 14;  
params.FontStyle = DrawText::Italic;  
params.Multiline = true;  
params.WordWrap = true;  
params.HAlignment = DrawText::Align_Justify;  
sz = pDm1->GetTextExtent(str, 200, &params);  
ptxt12 = *pDm1->CreateText(str, RectF(200, 300, sz), &params);  
ptxt12->SetColor(Render::Color(0, 0, 255, 130), 0, 1);
```



```

Render::Size<unsigned> viewSize = GetViewSize();
Render::Viewport dmViewport(viewSize.Width, viewSize.Height,
    int(viewSize.Width * GetSafeArea().Width),
    int(viewSize.Height * GetSafeArea().Height),
    int(viewSize.Width - 2 * viewSize.Width * GetSafeArea().Width),
    int(viewSize.Height - 2 * viewSize.Height * GetSafeArea().Height));
pDm1->SetViewport(dmViewport);
pDm1->Capture();
pRenderThread->AddDisplayHandle(pDm1->GetDisplayHandle(),
    FxRenderThread::DHCAT_Overlay, false);

```

```

Ptr<MovieDef> pmovieDef = *mLoader.CreateMovie("drawtext_fonts.swf",
    Loader::LoadAll);
pDm2 = *new DrawTextManager(pmovieDef);

```

DrawTextManager::GetHtmlExtent

// MovieDefのフォントを使って、HTMLテキストを作成

```

const wchar_t* html = L"<P>o123 <FONT FACE=\"Times New Roman\" SIZE
    =\"140\">\"L\"A<b><i><FONT
    COLOR='#3484AA'>6</FONT></i>пак</b>адабрА!</FONT></P>"
    L"<P><FONT FACE='Arial Unicode MS'>Hànyǔ; 华语/華語</FONT></P>"
    L"<P><FONT FACE='Batang'>한국어/조선말</FONT></P>"
    L"<P><FONT FACE='Symbol'>Privet!</FONT></P>";

```

```

Gfx::DrawTextManager::TextParams defParams2 = pDm2->GetDefaultTextParams();
defParams2.TextColor = Color(0xF0,0,0,0xFF); //red,alpha = 255
defParams2.Multiline = true;
defParams2.WordWrap = false;
SizeF htmlSz = pDm2->GetHtmlTextExtent(HtmlText, 0, &defParams2);
ptxt22 = *pDm2->CreateHtmlText(HtmlText, RectF(00, 100, htmlSz), &defParams2);

```

```

SizeF sz;
sz = pDm2->GetTextExtent("Animated");
ptxt21 = *pDm2->CreateText("Animated", RectF(600, 400, sz));
ptxt21->SetColor(Render::Color(0, 0, 255, 255));
Angle = 0;

```

```

SizeF sz;
Ptr<GFx::DrawText> pglowTxt
pglowTxt = *pDm2->CreateText("Glow", RectF(800, 350, SizeF(200, 220)));
pglowTxt->SetColor(Render::Color(120, 30, 192, 255));
pglowTxt->SetFontSize(72);
GFx::DrawText::Filter glowF(GFx::DrawText::Filter_Glow);
glowF.Glow.BlurX = glowF.Glow.BlurY = 2;
glowF.Glow.Strength = 1000;
glowF.Glow.Color = Render::Color(0, 0, 0, 255).ToColor32();
pglowTxt->SetFilters(&glowF);

```

```

pDm2->SetViewport(dmViewport);
pDm2->Capture();
pRenderThread->AddDisplayHandle(pDm2->GetDisplayHandle(),
                                FxRenderThread::DHCAT_Overlay, false);

```

```

void FxPlayerApp::OnUpdateFrame( bool needRepaint )
{
    DrawText::Matrix txt21matrix;
    Angle += 1;
    RectF r = ptxt21->GetRect();
    txt21matrix.AppendTranslation(-r.x1, -r.y1);
    txt21matrix.AppendRotation(Angle*3.14159f / 180.f);
    txt21matrix.AppendScaling(2);
    txt21matrix.AppendTranslation(r.x1, r.y1);
    ptxt21->SetMatrix(txt21matrix);

    pDm2->Capture();

    FxPlayerAppBase::OnUpdateFrame(needRepaint);
}

```

